Project Orange – ROADEF/EURO Challenge 2026 “Keep The Flow!”

Mô hình quy hoạch tuyến tính nguyên cho bài toán T-Adaptive Segment Routing

Họ và tên

25/10/2026

1 Giới thiệu

Bối cảnh. Orange vận hành nhiều mạng backbone IP/MPLS quy mô quốc gia và quốc tế, với hàng nghìn router, hàng nghìn liên kết và hàng chục nghìn luồng lưu lượng (*demand*) mỗi năm. Mục tiêu của kỹ thuật lưu lượng (*Traffic Engineering*) là dùng hạ tầng sẵn có hiệu quả hơn, chủ yếu để tránh nghẽn và giữ chất lượng dịch vụ. Theo quy tắc thiết kế chuẩn, mạng phải tiếp tục hoạt động khi một liên kết hay một nút không còn dùng được, kể cả khi điều đó là do nhà vận hành chủ động tắt để bảo trì.

Động lực. Theo cách truyền thống, gói tin đi theo đường ngắn nhất tính theo trọng số ω do người quản trị gán cho các liên kết. Cách này dễ triển khai, nhưng nhiều luồng thường dồn vào cùng vài liên kết, trong khi muốn điều chỉnh thì phải tìm lại cả bộ trọng số, một bài toán rất khó [5]. *Segment Routing* (SR) [4] khắc phục điều này: tại nút nguồn, một danh sách nút trung gian (*waypoint*) được ghi vào header của gói tin. Gói tin đi theo đường ngắn nhất tới waypoint thứ nhất, rồi tới waypoint tiếp theo, và cứ thế cho tới đích. Chỉ cần chọn waypoint là có thể lái lưu lượng tránh các liên kết nghẽn mà không phải sửa trọng số.

Khi một liên kết bị tắt để bảo trì, các đường ngắn nhất thay đổi và tình trạng nghẽn có thể xuất hiện ở chỗ khác. Ta có thể đổi waypoint giữa các chu kỳ, nhưng mỗi lần đổi là một thao tác vận hành tốn kém, nên tổng số thay đổi bị giới hạn bởi một *budget*.

Mô tả bài toán. Bài toán *T-Adaptive Segment Routing* (T-ASR) [1, 2] có dữ liệu vào gồm:

- một mạng có hướng với trọng số và dung lượng trên mỗi cung;
- một tập demand, mỗi demand có lưu lượng thay đổi theo chu kỳ thời gian;
- một lịch bảo trì cho biết những cung nào bị tắt ở mỗi chu kỳ;
- số segment tối đa của một đường;
- budget thay đổi giữa hai chu kỳ liên tiếp.

Yêu cầu là chọn, cho mỗi demand và mỗi chu kỳ, một danh sách waypoint sao cho tải của các liên kết là nhỏ nhất. Đề gốc dùng mục tiêu lexicographic trên toàn bộ vector tải. Báo cáo này dùng thành phần đầu tiên và quan trọng nhất của mục tiêu đó: *Maximum Link Utilization* (MLU). Bài toán là NP-khó và khó xấp xỉ, ngay cả trên topology rất đơn giản [6].

Nội dung báo cáo. Mục 2 trình bày mô hình MILP: dữ liệu, cách tính hệ số chia luồng, biến, mục tiêu và ràng buộc. Mục 3 trình bày thí nghiệm trên 20 instance SetA, trước tiên chỉ với chu kỳ $t = 0$ (giới hạn 1800 s), sau đó với hai chu kỳ $t \in \{0, 1\}$ có ràng buộc budget (giới hạn 3600 s).

2 Mô hình toán học

2.1 Dữ liệu

Bảng 1 liệt kê các tham số và nguồn lấy chúng. Mỗi instance gồm ba file JSON: `net` (mạng), `tm` (ma trận lưu lượng) và `scenario` (kịch bản bảo trì).

Ký hiệu	Ý nghĩa	Nguồn
$G = (V, A)$	đồ thị có hướng, $n = \|V\|$ nút, $\|A\|$ cung	`net`: `nodes`, `links`
$\omega(a)$	trọng số (metric) của cung a, chỉ dùng để tính đường ngắn nhất	`net`: `metric`
$c(a)$	dung lượng của cung a	`net`: `capacity`
D, (s_d, t_d)	tập demand, nguồn và đích của demand d	`tm`: `s`, `t`
$\nu(d, t)$	lưu lượng của demand d tại chu kỳ t	`tm`: `v[t]`
$T = \{0, \dots, h-1\}$	các chu kỳ, $T^* = T \setminus \{0\}$	`tm`: `num_time_slots`
$maxSeg$	số segment tối đa của một đường	`scenario`: `max_segments`
$q(t)$	tập cung bị tắt tại chu kỳ $t \in T^*$; $G_t = (V, A \setminus q(t))$	`scenario`: `interventions`
$\kappa(t)$	số thay đổi tối đa giữa chu kỳ $t-1$ và t	`scenario`: `budget`
$r(i, j, a, t)$	tỉ lệ luồng từ i đến j đi qua cung a trong G_t	**tính trước** (mục 2.2)

Bảng 1: Tham số của mô hình.

Tại $t = 0$ không có cung nào bị tắt và budget là vô hạn, nên chu kỳ 0 không xuất hiện trong file `scenario`. ID trong file JSON không nhất thiết liên tiếp hay theo thứ tự (ví dụ setA-14 liệt kê nút từ 249 về 0), nên chương trình tạo một bảng ánh xạ từ ID sang chỉ số $1..n$.

Segment path và segment. Một *segment path* $\langle s, w_1, \dots, w_\ell, t\rangle$ gồm nguồn s, các waypoint $w_1, \dots, w_\ell$ và đích t. Mỗi cặp nút liên tiếp $(s, w_1), (w_1, w_2), \dots, (w_\ell, t)$ là một *segment*. Luồng trên segment (i, j) đi theo các đường ngắn nhất từ i đến j, tức theo *forwarding graph* $FG(i, j)$. Khi có nhiều đường ngắn nhất, cơ chế ECMP chia đều luồng *tại mỗi nút* cho các cung ra của nút đó trong $FG(i, j)$.

2.2 Tính hệ số chia luồng $r(i, j, a, t)$

Hệ số $r(i, j, a, t)$ chỉ phụ thuộc vào G_t và ω, không phụ thuộc vào quyết định chọn waypoint, nên được tính trước như dữ liệu. Nhờ vậy tải là hàm *tuyến tính* của biến quyết định. Phải tính r cho *mọi* cặp (i, j), vì nút nào cũng có thể là waypoint. Cách tính như sau:

1. Chạy Dijkstra từ mọi nút để có d_{ij}, độ dài đường ngắn nhất từ i đến j trong G_t. Chương trình dùng `dijkstra_shortest_paths` của Graphs.jl.
2. Cung $a = (u, v)$ nằm trên một đường ngắn nhất từ i đến j khi và chỉ khi $d_{iu} + \omega(a) + d_{vj} = d_{ij}$.
3. Với mỗi đích j, xét các nút theo thứ tự $d_{\cdot j}$ giảm dần, đây là một thứ tự topo vì $\omega > 0$. Đặt một đơn vị luồng tại i. Tại mỗi nút u, chia đều lượng luồng đang có cho các cung ra (u, v) thoả $d_{uj} = \omega(u, v) + d_{vj}$. Lượng luồng đi qua cung a chính là $r(i, j, a, t)$.

So sánh với cách đếm đường ngắn nhất. Tài liệu hướng dẫn `calculating_r.pdf` đề xuất công thức $r_{ija} = \sigma_{iu}\sigma_{vj}/\sigma_{ij}$, trong đó σ là *số* đường ngắn nhất. Công thức này chia luồng đều cho từng *đường*, còn ECMP trong đề chia đều tại từng *nút*. Hai cách cho kết quả khác nhau khi các đường ngắn nhất tách ra rồi nhập lại. Ví dụ, với demand (v_0, v_5) của instance toy có ba đường ngắn nhất:

- theo ECMP: $r(a_{1,3}) = r(a_{1,4}) = 0.5$ và $r(a_{3,5}) = 0.75$, đúng như Table 2 của đề;
- theo cách đếm đường: $r(a_{1,3}) = 1/3$, $r(a_{1,4}) = 2/3$ và $r(a_{3,5}) = 2/3$.

Chúng tôi cài cả hai cách và dùng ECMP cho mô hình, vì cách này khớp với định nghĩa và ví dụ của đề. Trên SetA, hai cách cho cùng giá trị ở mọi cặp nút, trừ một cặp ở setA-18 và một cặp ở setA-20 (không phải cặp nguồn–đích của demand nào). Lý do là trong SetA, đường ngắn nhất hầu như luôn duy nhất.

2.3 Biến quyết định

Coi mỗi cặp nút (i, j), $i \neq j$, là một "cung ảo" của một đồ thị đầy đủ H trên V. Khi đó một segment path cho demand d chính là một đường đi từ s_d đến t_d trong H.

- $x_{ij}^{dt} \in \{0, 1\}$: bằng 1 nếu segment path của demand d tại chu kỳ t chứa segment (i, j). Chỉ tạo biến cho các cặp (i, j) có đường đi từ i tới j trong G_t; các cặp khác không dùng được, nên coi $x_{ij}^{dt} = 0$.
- $U \geq 0$: MLU, tức tải lớn nhất trên mọi cung và mọi chu kỳ.
- $y_{ij}^{dt} \geq 0$ (chỉ khi có budget): biến phụ để tuyến tính hoá $|x_{ij}^{dt} - x_{ij}^{d,t-1}|$.

2.4 Hàm mục tiêu

$$\min\ U \qquad \text{với} \qquad \lambda(a, t) \leq U \quad \forall t \in T,\ a \in A \setminus q(t). \tag{1}$$

Tải $\lambda(a, t)$ là tỉ số giữa lưu lượng đi qua cung a và dung lượng $c(a)$. Mục tiêu lexicographic của đề cực tiểu tải lớn nhất, rồi tải lớn thứ hai, và cứ thế tiếp tục. Thành phần đầu tiên của nó chính là $\min U$, nên mọi nghiệm tối ưu lexicographic đều tối ưu theo MLU [2]. Điều ngược lại không đúng: hai nghiệm có cùng MLU có thể khác nhau ở các tải nhỏ hơn.

2.5 Ràng buộc

(1) Bảo toàn luồng. Mỗi demand đi trên đúng một segment path từ nguồn tới đích:

$$\sum_{j \in V \setminus \{i\}} x_{ij}^{dt} - \sum_{j \in V \setminus \{i\}} x_{ji}^{dt} = \begin{cases} 1 & i = s_d \\ -1 & i = t_d \\ 0 & \text{khác} \end{cases} \qquad \forall i \in V,\ d \in D,\ t \in T. \tag{2}$$

Tại nguồn, số segment đi ra nhiều hơn số đi vào đúng một; tại đích thì ngược lại; tại mỗi waypoint, số segment đi vào bằng số đi ra.

(2) Số segment. Giới hạn độ dài header, tính cả segment đầu (từ nguồn) và segment cuối (tới đích):

$$\sum_{i,j \in V} x_{ij}^{dt} \leq maxSeg \qquad \forall d \in D,\ t \in T. \tag{3}$$

(3) Tải. Lưu lượng trên cung a là tổng, trên mọi demand và mọi segment được chọn, của phần luồng đi qua a:

$$\sum_{d \in D} \sum_{i,j \in V} r(i, j, a, t)\, \nu(d, t)\, x_{ij}^{dt} \ \leq\ U\, c(a) \qquad \forall t \in T,\ a \in A \setminus q(t). \tag{4}$$

Vì r và ν là hằng số nên ràng buộc này tuyến tính. Các cung trong $q(t)$ có $r = 0$ vì r được tính trên G_t.

(4) Budget. Số segment bị thêm hoặc bị bỏ giữa hai chu kỳ liên tiếp không vượt quá $\kappa(t)$:

$$\sum_{d \in D} \sum_{i \neq j} \left| x_{ij}^{dt} - x_{ij}^{d,t-1} \right| \leq \kappa(t) \qquad \forall t \in T^*. \tag{5}$$

Trị tuyệt đối không tuyến tính, nên ta thay (4) bằng

$$y_{ij}^{dt} \geq x_{ij}^{dt} - x_{ij}^{d,t-1}, \qquad y_{ij}^{dt} \geq x_{ij}^{d,t-1} - x_{ij}^{dt}, \qquad \sum_{d} \sum_{i \neq j} y_{ij}^{dt} \leq \kappa(t). \tag{6}$$

Hai bất đẳng thức đầu buộc $y_{ij}^{dt} \geq |x_{ij}^{dt} - x_{ij}^{d,t-1}|$, nên ràng buộc thứ ba kéo theo (4). Ngược lại, mọi nghiệm thoả (4) đều mở rộng được thành một nghiệm của hệ trên bằng cách đặt y bằng đúng trị tuyệt đối. Segment nào có đường đi ở chu kỳ này mà không có ở chu kỳ kia thì biến tương ứng được coi là 0. Ví dụ, đổi $\langle s, w, t \rangle$ thành $\langle s, t \rangle$ tốn 3 đơn vị: bỏ (s, w) và (w, t), thêm (s, t). Mỗi demand đổi đường tốn ít nhất 2 đơn vị, vì hiệu $x^t - x^{t-1}$ là một luồng vòng khác 0. Do đó $\kappa(t) = 1$ nghĩa là không demand nào được đổi đường.

Tóm tắt. Mô hình cho chu kỳ $t = 0$ (Step 5) gồm mục tiêu $\min U$ với các ràng buộc (1)–(3) và $x \in \{0, 1\}$. Mô hình cho hai chu kỳ (Step 8) gồm (1)–(3) cho mỗi $t \in \{0, 1\}$, cộng thêm (4) đã tuyến tính hoá.

2.6 Ví dụ kiểm tra: instance toy

Instance toy của đề có 7 nút, 22 cung và 2 demand, (v_0, v_5) và (v_2, v_5), với $maxSeg = 4$. Tại $t = 1$ các cung $a_{1,3}$ và $a_{1,4}$ bị tắt, và $\kappa(1) = 20$.

- Khi không có waypoint, MLU tại $t = 0$ là 0.4375, đạt trên cung $a_{3,5}$.
- Mô hình Step 5 trả về MLU tối ưu 0.375 với segment path $\langle v_0, v_4, v_5\rangle$, đúng nghiệm minh hoạ trong Figure 4 của đề.
- Mô hình Step 8 trả về MLU 0.375 cho cả hai chu kỳ, giữ nguyên waypoint nên dùng 0 thay đổi.

Cả hai giá trị đã được xác nhận bằng cách vét cạn mọi danh sách waypoint.

2.7 Kích thước mô hình

Số biến nhị phân cho mỗi chu kỳ xấp xỉ $|D| \cdot n(n-1)$. Ràng buộc (3) có khoảng $|D| \cdot \mathrm{nnz}(r)$ hệ số khác 0, với $\mathrm{nnz}(r)$ là số cặp (segment, cung) có $r > 0$. Trên SetA, số biến nhị phân đi từ $1.5 \cdot 10^4$ (setA-01) tới $9.6 \cdot 10^8$ (setA-20) cho mỗi chu kỳ (Bảng 3). Đây là lý do mô hình chỉ dựng được cho các instance nhỏ trên máy thí nghiệm (mục 3).

2.8 Phương pháp 2: mô hình theo đường đi và sinh cột

Để giải được cả các instance lớn, ta viết lại mô hình theo đường đi. Gọi $\mathcal{P}_d$ là tập mọi segment path hợp lệ của demand d (tối đa $maxSeg$ segment, không dùng một segment hai lần), và $L_t(p, a) = \sum_{(i,j)\in p} r(i, j, a, t)$ là phần luồng mà path p đặt lên cung a. Với một chu kỳ (Step 5):

$$\min U \quad \text{v.đ.k.} \quad \sum_{p\in\mathcal{P}_d} \lambda_{dp} = 1 \ \ \forall d, \qquad \sum_{d} \sum_{p\in\mathcal{P}_d} \nu(d, 0)\, L_0(p, a)\, \lambda_{dp} \le U\, c(a) \ \ \forall a, \qquad \lambda_{dp} \in \{0, 1\}. \quad (7)$$

Chọn $\lambda_{dp} = 1$ tương đương với đặt $x^{d0}_{ij} = 1$ cho các segment của p, nên hai mô hình có cùng giá trị tối ưu. Số cột rất lớn, nên ta sinh dần (column generation):

- **Bài toán chính thu hẹp**: nới lỏng LP chỉ với các path đã sinh. Ban đầu mỗi demand chỉ có path đi thẳng $\langle s_d, t_d\rangle$.
- **Bài toán con**: gọi $w_a \ge 0$ là đối ngẫu của ràng buộc tải và μ_d là đối ngẫu của ràng buộc chọn path. Path nào có reduced cost $\nu(d) \sum_a L(p, a) w_a - \mu_d < 0$ thì được thêm vào. Chi phí cộng được theo segment, $W(i, j) = \sum_a r(i, j, a) w_a$, nên đây là bài toán đường đi ngắn nhất có tối đa $maxSeg$ cung trên đồ thị đầy đủ H. Ta giải bằng quy hoạch động kiểu Bellman–Ford, $O(maxSeg \cdot n^2)$ cho mỗi nguồn [7].
- **Cận dưới**: với mọi $w \ge 0$, $LB(w) = \sum_d \nu(d) \min_p \sum_a L(p, a) w_a \,/\, \sum_a c(a) w_a$ là cận dưới hợp lệ (cận Lagrange) và có ngay ở mọi vòng. Ta lấy giá trị lớn hơn giữa nó và cận "một demand" $\max_d \min_p \max_a \nu(d) L(p, a)/c(a)$. Cận sau quan trọng khi có ít demand lớn.
- **Hai chu kỳ (Step 8)**: một cột là cặp path (p_0, p_1). Chi phí budget của nó là số segment có ở đúng một trong hai path, $\Delta(p_0, p_1) = |S(p_0) \triangle S(p_1)|$, với $S(p)$ là tập segment của p. Khi đó (4) là một ràng buộc tuyến tính chính xác $\sum \Delta\, \lambda \le \kappa$, không cần biến y, và hai chu kỳ được tối ưu đồng thời. Bài toán con xét path chung cho cả hai chu kỳ và cặp path tốt nhất của từng chu kỳ. Hai path có tập segment khác nhau thì khác nhau ít nhất 2 segment, còn hai path có cùng tập segment đặt cùng một tải, nên cận Lagrange vẫn tính được.
- **Nghiệm nguyên**: giải MILP (HiGHS) chỉ trên các path đã sinh, rồi lặp lại: tìm các cung có tải $\ge 99.5\%$ MLU; sinh path thay thế cho các demand đi qua chúng bằng một quy hoạch động "cổ chai" (cực tiểu tải lớn nhất trên đường đi, giữ nguyên các demand khác); rồi giải lại MILP.

Gap được tính như ở phương pháp 1, $1 - LB/UB$, với LB là cận dưới ở trên. "Optimal" nghĩa là gap $\leq 10^{-4}$, cùng ngưỡng `mip_rel_gap` mặc định của HiGHS mà phương pháp 1 dùng; tức tối ưu đã được chứng minh, sai khác chỉ do làm tròn.

2.9 Điểm mới so với mô hình gốc của đề

Bảng 2 tóm tắt những điểm phương pháp 2 khác mô hình (1)–(4) của đề.

Khía cạnh	Mô hình gốc (phương pháp 1)	Phương pháp 2 (sinh cột)
Biến quyết định	$x_{ij}^{dt} \in \{0,1\}$ cho mỗi demand và mỗi cặp nút, khoảng $\lvert D\rvert \cdot n(n-1)$ biến mỗi chu kỳ (tới $9.6 \cdot 10^8$)	λ_{dp} cho mỗi (demand, segment path); ràng buộc (1), (2) nằm sẵn trong định nghĩa path hợp lệ
Kích thước	dựng toàn bộ mô hình từ đầu; thiếu RAM từ setA-10	chỉ sinh các path có reduced cost âm: 25 đến khoảng 23 000 cột
Cận dưới	cận LP của branch-and-bound, chỉ có khi giải được	cận Lagrange ở mọi vòng và cận "một demand" (DP cổ chai)
Budget (Step 9)	tuyến tính hoá $\lvert x^t - x^{t-1}\rvert$ bằng biến phụ y	cột là cặp (p_0, p_1) với hệ số $\Delta = \lvert S(p_0) \triangle S(p_1)\rvert$; ràng buộc chính xác, không cần y
Hai chu kỳ	tối ưu đồng thời trong một MILP lớn	vẫn tối ưu đồng thời (min của max), khác với cách giải tuần tự $t=0$ rồi $t=1$ của [3]
Nghiệm nguyên	branch-and-bound trên toàn bộ biến x	làm tròn LP $\rightarrow$ MILP trên tập path đã sinh $\rightarrow$ vòng cải thiện quanh link nghẽn
Kiểm tra nghiệm	giá trị của solver	tính lại tải trong Julia và bằng một script Python độc lập, định dạng checker chính thức

Bảng 2: So sánh phương pháp 2 với mô hình gốc của đề.

Cụ thể, các ý mới chính là:

1. **Biến theo đường đi.** Thay vì quyết định từng segment, ta chọn cả một segment path. Mô hình chỉ còn ràng buộc chọn path và ràng buộc tải, với $\lvert D\rvert + \lvert A\rvert$ dòng, và có cùng giá trị tối ưu với mô hình gốc.
2. **Sinh cột.** Path mới được tìm bằng bài toán đường ngắn nhất có tối đa $maxSeg$ cung trên đồ thị segment H, với giá cung $W(i,j) = \sum_a r(i,j,a) w_a$ lấy từ đối ngẫu. Một lần quy hoạch động từ mỗi nguồn cho path rẻ nhất tới mọi đích; với setA-20 ($n = 400$), một vòng pricing mất dưới 1 s.
3. **Cận dưới chứng minh được.** Cận Lagrange có ngay ở mọi vòng nhờ kết quả pricing, kể cả khi chưa hội tụ. Cận "một demand" bù cho trường hợp ít demand lớn: ví dụ ở Step 9 setA-02, LP chỉ cho 0.624 nhưng cận này bằng 0.903075, đúng MLU tìm được.
4. **Budget chính xác bằng cột cặp path.** Mỗi thay đổi đường tốn ít nhất 2 segment, nên cận Lagrange dùng $\min(\text{chung}_d,\ \text{riêng}_d + 2\rho)$. Trường hợp $\kappa = 1$ (setA-05, 10, 17) được xử lý tự động: chỉ cột $\Delta = 0$ vừa budget. Cách viết "MILP cặp" chặt hơn nhiều so với tuyến tính hoá bằng y: ở setA-10, MILP giải xong trong 10 s thay vì quá 300 s.
5. **Vòng cải thiện quanh link nghẽn.** Tập cột sinh ra để giải tốt LP, chưa chắc chứa path cần cho nghiệm nguyên tốt nhất (ở toy, MILP trên tập cột chỉ đạt 0.4375 trong khi tối ưu là 0.375). Vì vậy ta lặp: tìm link có tải $\geq 99.5\%$ MLU; sinh path mới cho các demand đi qua chúng bằng DP "cổ chai", giữ nguyên tải của các demand khác, kể cả path tránh hẳn link nghẽn; rồi giải lại MILP chỉ cho một phần demand được đổi đường. Khi bị kẹt, các ngưỡng được nới rộng.

Ý tưởng sinh cột cho Segment Routing đã có trong [7] và đã được áp dụng trên SetA trong [3]. Phần riêng của chúng tôi là cột cặp path với budget chính xác và tối ưu đồng thời hai chu kỳ, cận "một demand", MILP cặp không cần biến y, và vòng cải thiện quanh link nghẽn.

3 Thí nghiệm số

3.1 Mô tả instance

SetA gồm 20 instance (Bảng 3). Mọi instance đều có $h = 2$ chu kỳ, $maxSeg = 6$, và đúng một cung bị tắt tại $t = 1$. Với mọi cung $u \to v$ đều có cung ngược $v \to u$, và mỗi cặp nguồn–đích chỉ có một demand. Mười instance có cờ `multigraph` trong file JSON (05, 06, 08, 10, 12, 13, 14, 16, 17, 19), nhưng không instance nào có cung song song. Cột MLU_0 là MLU khi mọi demand đi thẳng theo đường ngắn nhất (không waypoint), lấy max trên hai chu kỳ. Giá trị này xấp xỉ 1 ở đa số instance, tức routing mặc định làm liên kết nặng nhất đầy đúng 100%.

Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	$\kappa(1)$	#biến x (t=0)	Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	$\kappa(1)$	#biến x (t=0)
01	20	80	40	51	$1.5 \cdot 10^4$	11	200	1000	400	89	$1.6 \cdot 10^7$
02	30	150	45	63	$3.9 \cdot 10^4$	12	200	898	400	13	$1.6 \cdot 10^7$
03	50	250	20	53	$4.9 \cdot 10^4$	13	200	1000	2000	12	$8.0 \cdot 10^7$
04	50	250	200	44	$4.9 \cdot 10^5$	14	250	1108	600	13	$3.7 \cdot 10^7$
05	100	396	100	1	$9.9 \cdot 10^5$	15	250	1250	600	54	$3.7 \cdot 10^7$
06	100	500	500	13	$5.0 \cdot 10^6$	16	250	1452	4800	13	$3.0 \cdot 10^8$
07	100	500	800	90	$7.9 \cdot 10^6$	17	300	1270	2000	1	$1.8 \cdot 10^8$
08	150	654	200	13	$4.5 \cdot 10^6$	18	300	1500	2000	89	$1.8 \cdot 10^8$
09	150	750	200	18	$4.5 \cdot 10^6$	19	300	1998	6000	13	$5.4 \cdot 10^8$
10	150	966	1000	1	$2.2 \cdot 10^7$	20	400	2000	6000	90	$9.6 \cdot 10^8$

Bảng 3: Các instance SetA. #biến x là số biến nhị phân của mô hình Step 5.

3.2 Môi trường và thiết lập

- **Máy**: Intel Xeon 2.20 GHz (8 nhân, 16 luồng), 62.8 GB RAM, Linux; mọi thí nghiệm chạy trong Docker (image `orange-tasr`) để tái lập được.
- **Phần mềm**: Julia 1.13.2, JuMP 1.32.1, Graphs.jl 1.15.0, solver HiGHS 1.15.1 (HiGHS.jl 1.26.1).
- **Giới hạn thời gian**: 1800 s cho Step 6 và 3600 s cho Step 9. *CPU time* là thời gian của riêng lời gọi `optimize!`, lấy bằng `solve_time`.
- **Thiết lập solver**: tham số `mip_heuristic_effort` được đặt bằng 0.3 (mặc định là 0.05), vì thử nghiệm cho thấy cận dưới LP gần như luôn chặt, còn việc tìm nghiệm khả thi mới là phần chậm. Solver được cho một nghiệm khởi đầu là "không waypoint". Nghiệm này luôn khả thi, nên mọi lần chạy đều có cận trên.
- **Gap**: $1 - LB/UB$, với LB là cận dưới tốt nhất (`objective_bound`) và UB là giá trị nghiệm tốt nhất.
- **Số lần chạy**: HiGHS là deterministic, nên mỗi lần chạy dùng một `random_seed` khác $(0, \ldots, 9)$.
  - Instance giải tối ưu trong dưới 300 s được chạy tối đa 10 lần (dừng sớm nếu lần chạy tiếp theo có thể vượt chốt chặn cứng); bảng ghi trung bình.
  - Instance chạm giới hạn thời gian chỉ chạy 1 lần, vì 10 lần $\times$ 1800–3600 s vượt quá thời gian có được.
- **Giới hạn bộ nhớ**: mô hình có hơn $5 \cdot 10^6$ biến không được dựng, vì riêng việc dựng mô hình $4.5 \cdot 10^6$ biến trong JuMP đã tốn 3.3 GB RAM, và HiGHS cần thêm một bản sao khi giải.
- **Ràng buộc bổ sung ở Step 6**: với một chu kỳ, thêm ràng buộc mỗi segment path đi qua mỗi nút tối đa một lần ($\sum_j x^{d0}_{ji} \le 1$, không quay lại nguồn, không đi tiếp từ đích). Ràng buộc này không làm thay đổi giá trị tối ưu, vì bỏ một vòng lặp chỉ làm tải giảm ($r \ge 0$), nhưng làm mô hình chặt hơn và giải nhanh hơn nhiều (setA-02: 364 s xuống 5 s). Ở Step 9 không dùng, vì khi có budget, đường có vòng lặp có thể cần cho nghiệm tối ưu [2].

- **Chốt chặn cứng**: với mô hình lớn, HiGHS có thể vượt giới hạn thời gian, vì nó không kiểm tra giới hạn trong lúc presolve và giải LP gốc. Vì vậy mỗi instance chạy trong một tiến trình riêng, và bị dừng nếu vượt giới hạn thêm 900 s.
- **Phương pháp 2** (mục 2.8): giới hạn 600 s cho sinh cột, 300 s cho MILP trên tập cột và 780 s (Step 6) hoặc 2400 s (Step 9) cho vòng cải thiện, tổng nằm dưới 1800 s và 3600 s. Mỗi instance chạy 10 lần với `random_seed` của HiGHS bằng $0, \ldots, 9$. Phần sinh cột cho cùng kết quả mọi lần, nên khác biệt giữa các lần chỉ đến từ các MILP chạm giới hạn thời gian. 10 lần chạy được thực hiện 8 lượt song song, mỗi lượt 2 luồng. *CPU time* là thời gian thực của toàn bộ phần giải (sinh cột, MILP và vòng cải thiện), không tính đọc dữ liệu và tính r. Số nút là tổng số nút branch-and-bound của các MILP trên tập cột.

3.3 Kết quả với chu kỳ $t = 0$ (Step 6)

Bảng 4 ghi, cho mỗi instance:

- MLU khi không có waypoint (MLU_0);
- giá trị tốt nhất tìm được (Obj);
- CPU time trung bình (s), trạng thái và gap;
- số nút trung bình của cây branch-and-bound, và số lần chạy.

Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	MLU_0	Obj	CPU (s)	Status	Gap	#nút	Lần
01	20	80	40	1.0000	0.9294	4.5	Optimal	0	1	10
02	30	150	45	1.0000	0.5486	318.0	Optimal	0	0	1
03	50	250	20	1.0000	0.9435	58.2	Optimal	0	1	9
04	50	250	200	0.6918	0.6918	1800.3	Not Opt.	33.64%	0	1
05	100	396	100	0.3333	0.1155	1706.0	Optimal	0	1	1
07	100	500	800	0.9080	*mô hình quá lớn (7.9 triệu biến)*					
10	150	966	1000	1.0000	*mô hình quá lớn (22.4 triệu biến)*					
11	200	1000	400	1.0000	*mô hình quá lớn (15.9 triệu biến)*					
12	200	898	400	0.4400	*mô hình quá lớn (15.9 triệu biến)*					
13	200	1000	2000	1.0000	*mô hình quá lớn (79.6 triệu biến)*					
14	250	1108	600	1.0000	*mô hình quá lớn (37.4 triệu biến)*					
15	250	1250	600	1.0000	*mô hình quá lớn (37.4 triệu biến)*					
16	250	1452	4800	0.9672	*mô hình quá lớn (298.8 triệu biến)*					
17	300	1270	2000	1.0001	*mô hình quá lớn (179.4 triệu biến)*					
18	300	1500	2000	0.6334	*mô hình quá lớn (179.4 triệu biến)*					
19	300	1998	6000	1.0000	*mô hình quá lớn (538.2 triệu biến)*					
20	400	2000	6000	1.0000	*mô hình quá lớn (957.6 triệu biến)*					

* CPU time vượt giới hạn: HiGHS không dừng được trong lúc presolve hoặc giải LP gốc. "–" ở cột Gap: không có cận dưới. "Bị dừng": vượt chốt chặn cứng, Obj là MLU của nghiệm khởi đầu.

Bảng 4: Kết quả Step 6: chỉ chu kỳ $t = 0$, giới hạn 1800 s.

3.4 Kết quả với hai chu kỳ $t \in \{0, 1\}$ và budget (Step 9)

3.5 Kết quả của phương pháp 2 (sinh cột)

Bảng 6 ghi kết quả của phương pháp 2 (mục 2.8) trên cả 20 instance, trung bình 10 lần chạy. Cột "Tốt nhất" là MLU tốt nhất mà 53 đội đạt được ở vòng sprint của challenge trên cùng bài toán hai chu kỳ [9].

Nhận xét:

- Phương pháp 2 giải được **cả 20 instance** trong giới hạn thời gian. Lần chạy lâu nhất mất 1097 s ở Step 6 và 1225 s ở Step 9 (trung bình lâu nhất: 841 s và 1140 s, đều ở setA-16). Mỗi lần chạy chỉ dùng khoảng 1.3–2 GB RAM, vì chỉ sinh vài chục đến khoảng 23 000 cột thay vì tới $9.6 \cdot 10^8$ biến.

Inst	$\|V\|$	$\|A\|$	$\|D\|$	MLU_0	Obj	CPU (s)	Status	Gap	#nút	Lần
01	20	80	40	1.0000	0.9294	163.9	Optimal	0	1	5
02	30	150	45	1.0981	0.9031	172.4	Optimal	0	1	4
03	50	250	20	1.0000	0.9435	704.2	Optimal	0	1	1
04	50	250	200	1.0000	1.0000	9905.0*	Not Opt.	–	0	1
05	100	396	100	1.0000	1.0000	>10800	Bị dừng	–	–	1
06	100	500	500	1.0000	*mô hình quá lớn (14.8 triệu biến)*					
07	100	500	800	1.0000	*mô hình quá lớn (23.8 triệu biến)*					
08	150	654	200	1.0000	*mô hình quá lớn (13.4 triệu biến)*					
09	150	750	200	1.0000	*mô hình quá lớn (13.4 triệu biến)*					
10	150	966	1000	1.0000	*mô hình quá lớn (67.0 triệu biến)*					
11	200	1000	400	1.0000	*mô hình quá lớn (47.8 triệu biến)*					
12	200	898	400	0.7201	*mô hình quá lớn (47.8 triệu biến)*					
13	200	1000	2000	1.0000	*mô hình quá lớn (238.8 triệu biến)*					
14	250	1108	600	1.0000	*mô hình quá lớn (112.0 triệu biến)*					
15	250	1250	600	1.0000	*mô hình quá lớn (112.0 triệu biến)*					
16	250	1452	4800	1.0000	*mô hình quá lớn (896.4 triệu biến)*					
17	300	1270	2000	1.0001	*mô hình quá lớn (538.2 triệu biến)*					
18	300	1500	2000	1.0000	*mô hình quá lớn (538.2 triệu biến)*					
19	300	1998	6000	1.0000	*mô hình quá lớn (1614.6 triệu biến)*					
20	400	2000	6000	1.0000	*mô hình quá lớn (2872.8 triệu biến)*					

* CPU time vượt giới hạn: HiGHS không dừng được trong lúc presolve hoặc giải LP gốc. "–" ở cột Gap: không có cận dưới. "Bị dừng": vượt chốt chặn cứng, Obj là MLU của nghiệm khởi đầu.

Bảng 5: Kết quả Step 9: hai chu kỳ $t \in \{0, 1\}$ có ràng buộc budget, giới hạn 3600 s. Obj là MLU trên cả hai chu kỳ.

- Ở Step 9, giá trị tốt nhất qua 10 lần chạy **bằng kết quả tốt nhất của challenge ở cả 20 instance** (sai khác 10^{-6} do làm tròn). Ở 19/20 instance, cả 10 lần chạy đều đạt giá trị đó; ở setA-19, 9/10 lần đạt và 1 lần dừng ở 0.048712. MLU trung bình trên 20 instance giảm từ 0.989 (phương pháp 1, phần lớn là nghiệm không waypoint) xuống 0.572; ở Step 6, từ 0.839 xuống 0.481.
- Tối ưu được chứng minh (gap $\leq 10^{-4}$) ở 12/20 instance của Step 6 và 13/20 instance của Step 9 (thêm setA-19 ở 9/10 lần chạy). Phần gap còn lại chủ yếu do cận dưới yếu chứ không do nghiệm kém. Ví dụ setA-03 có gap 32.8% nhưng phương pháp 1 đã chứng minh 0.943543 là tối ưu: instance này chỉ có 20 demand, trong đó có demand lớn, nên nới lỏng LP (cho phép chia một demand ra nhiều đường) cho cận thấp hơn nhiều.
- Hai phương pháp bổ sung cho nhau. Phương pháp 1 là mô hình gốc và chứng minh được tối ưu ở instance nhỏ. Phương pháp 2 mở rộng tới instance lớn và cho cận dưới để đánh giá nghiệm.

4 Kết luận và hướng phát triển

Chúng tôi đã cài đặt mô hình MILP của đề cho bài toán T-ASR bằng Julia/JuMP với solver HiGHS, gồm:

- phần đọc dữ liệu;
- tính hệ số chia luồng ECMP;
- mô hình một chu kỳ;
- mô hình hai chu kỳ có budget đã tuyến tính hoá.

Trên instance toy, mô hình cho đúng nghiệm minh hoạ của đề. Hạn chế chính là kích thước: số biến tăng theo $|D| \cdot n^2$, nên mô hình đầy đủ chỉ dựng được cho các instance nhỏ. Để vượt qua hạn chế này, chúng tôi cài thêm phương pháp sinh cột trên mô hình theo đường đi (mục 2.8). Phương pháp này giải được cả 20 instance trong giới hạn thời gian. Ở bài toán hai chu kỳ, giá trị tốt nhất qua 10 lần chạy bằng đúng MLU tốt nhất của

Inst	Step 6 ($t = 0$, giới hạn 1800 s)					Step 9 ($t \in \{0, 1\}$, giới hạn 3600 s)					
	Obj	CPU (s)	Status	Gap	#nút	Obj	CPU (s)	Status	Gap	#nút	Tốt nhất
01	0.9294	0.0	Opt.	0	0	0.9294	2.3	Opt.	0	0	0.9294
02	0.5486	0.1	Opt.	0	3	0.9031	0.0	Opt.	0	1	0.9031
03	0.9435	0.1	N.Opt.	32.84%	4	0.9435	1.1	N.Opt.	32.84%	4	0.9435
04	0.4591	0.5	N.Opt.	0.01%	265	0.5812	4.8	N.Opt.	0.02%	9102	0.5812
05	0.1155	0.0	Opt.	0	0	0.2050	0.0	Opt.	0	0	0.2050
06	0.0986	0.6	Opt.	0	17	0.0986	3.1	Opt.	0	7	0.0986
07	0.9080	0.1	Opt.	0	0	0.9080	0.1	Opt.	0	0	0.9080
08	0.3189	0.1	Opt.	0	2	0.3189	0.3	Opt.	0	3	0.3189
09	0.5054	0.1	Opt.	0	1	0.8497	0.1	Opt.	0	0	0.8497
10	0.0696	7.8	N.Opt.	1.79%	393	0.0717	22.6	N.Opt.	4.76%	488	0.0717
11	0.6446	0.5	N.Opt.	4.88%	10	0.7858	0.2	Opt.	0	0	0.7858
12	0.6401	0.2	N.Opt.	9.55%	3	0.8799	0.2	Opt.	0	0	0.8799
13	0.0389	28.2	N.Opt.	0.10%	8917	0.0410	366.6	N.Opt.	0.57%	11782	0.0410
14	0.4342	0.4	N.Opt.	2.37%	4	0.5176	1.0	N.Opt.	1.97%	22	0.5176
15	0.8252	0.2	Opt.	0	0	0.8987	0.4	Opt.	0	0	0.8987
16	0.0389	841.4	N.Opt.	0.88%	64662	0.0443	1139.5	N.Opt.	0.46%	9755	0.0443
17	0.4242	0.5	Opt.	0	0	0.4242	0.9	Opt.	0	0	0.4242
18	0.6334	0.4	Opt.	0	0	1.0000	0.8	Opt.	0	0	1.0000
19	0.0468	409.0	Opt.	0	4921	0.0470	860.6	Optimal 9/10	0.38%	4004	0.0468
20	0.9913	4.3	Opt.	0	0	0.9913	6.2	Opt.	0	0	0.9913

Bảng 6: Phương pháp 2: trung bình 10 lần chạy (`random_seed` $0, \ldots, 9$). Opt. = gap $\leq 10^{-4}$, N.Opt. = chưa chứng minh tối ưu. "Tốt nhất": MLU tốt nhất của vòng sprint ROADEF/EURO 2026.

vòng sprint challenge ở cả 20 instance (mục 3.5), và tối ưu được chứng minh ở 13 instance. Các điểm mới so với mô hình gốc được tóm tắt ở mục 2.9. Các hướng phát triển tiếp:

- làm chặt cận dưới khi có ít demand lớn, ví dụ bằng branch-and-price thay cho việc giải MILP trên tập cột cố định;
- tiền xử lý để loại các segment path bị trội [8];
- với mục tiêu lexicographic đầy đủ của challenge, dùng chuỗi MILP như STELA hoặc ALEXA [2], có thể trên tập cột đã sinh.

Tài liệu

[1] A. Benhamiche, Y. Carlinet, M. Chopin, E. Gourdin, N. Perrot. *Challenge EURO/ROADEF 2026 – Problem Description "Keep The Flow!"*. Orange Research, 2026.

[2] A. Benhamiche, K. Bouaachra, Y. Carlinet, M. Chopin, E. Gourdin, N. Perrot. On the T-Adaptive Segment Routing Problem. arXiv:2609.32896, 2026.

[3] B. Jaumard, N. P. Tran, J. M. Ziazet. Segment Routing Traffic Engineering with Time-Based Reconfiguration Constraints. arXiv:2610.04759, 2026.

[4] C. Filsfils et al. Segment Routing Architecture. RFC 8402, 2018.

[5] B. Fortz, M. Thorup. Internet traffic engineering by optimizing OSPF weights. IEEE INFOCOM, 2000.

[6] C. Bazgan, M. Chopin, A. Nichterlein, C. Richer. Parameterized complexity of segment routing. IEEE INFOCOM, 2025.

[7] M. Jadin, F. Aubry, P. Schaus, O. Bonaventure. CG4SR: Near optimal traffic engineering for segment routing with column generation. IEEE INFOCOM, 2019.

[8] H. Callebaut, J. De Boeck, B. Fortz. Preprocessing for segment routing optimization. *Networks* 82(4):459–478, 2023.

[9] Orange Open Source. EURO/ROADEF Challenge 2026: dữ liệu, checker và kết quả vòng sprint. https://gitlab.com/Orange-OpenSource/network-optimization-tools/challenge-roadef-2026, 2026.